

TECHNICAL PRESENTATION

Application of Queue Data Structure in Ticket Counter Management

Presented by: B. Siva Parvathi

Reg no:192525194

Department: Artificial Intelligence & Machine Learning

Course: Data Structures(CSA0302)

INTRODUCTION

- A Queue is a linear data structure that follows the FIFO (First In, First Out) principle.
- In a ticket counter, passengers wait in a line to receive their boarding passes.
- The passenger who arrives first is served first.
- New passengers are added at the rear, while passengers are served and removed from the front.
- Therefore, a Queue is an efficient and suitable data structure for managing passengers in an organized and fair manner.



PROBLEM STATEMENT

Ticket Counter Scenario

- Passengers wait in a line to get their boarding passes.
- The passenger who arrives first should be served first.
- New passengers join at the **rear**.
- Served passengers leave from the **front**.

Example:

FRONT $\rightarrow$ P1 $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ P4 $\leftarrow$ REAR

Service Order: P1 $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ P4



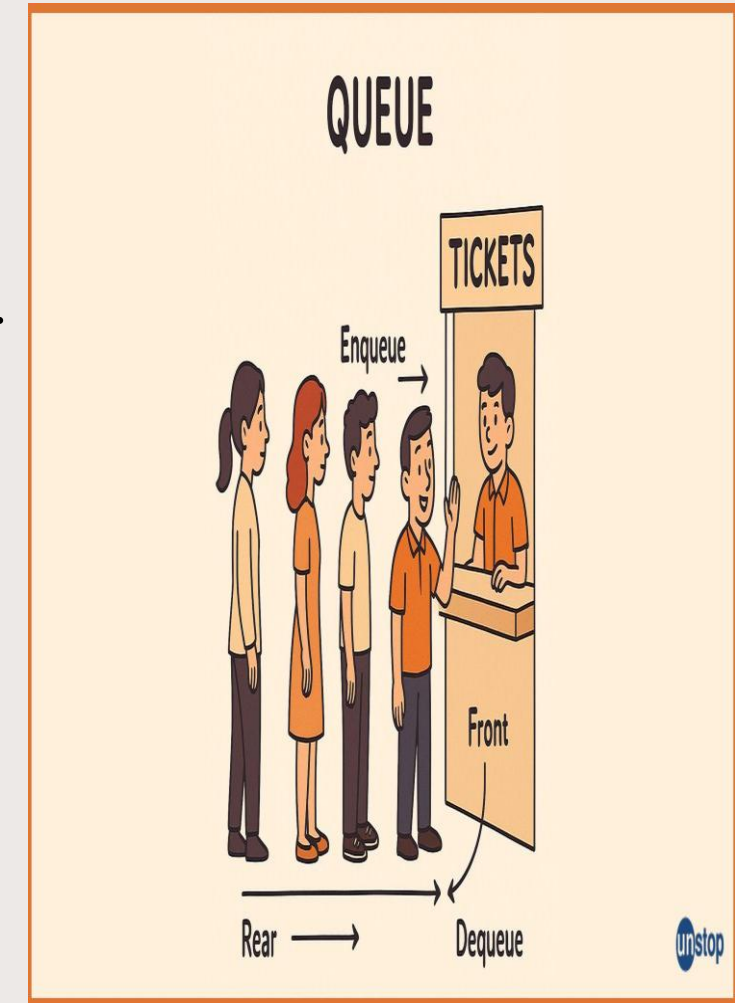
PROPOSED DATA STRUCTURE: QUEUE

Why Queue?

- A **Queue** is a linear data structure.
- It follows the **FIFO (First In, First Out)** principle.
- It is the best choice for managing passengers waiting in line.
- It maintains the correct order of service.

Main Operations:

- **Enqueue** → Add passenger at Rear
- **Dequeue** → Remove passenger from Front
- **Peek** → View the Front passenger



EFFICIENCY & ADVANTAGES

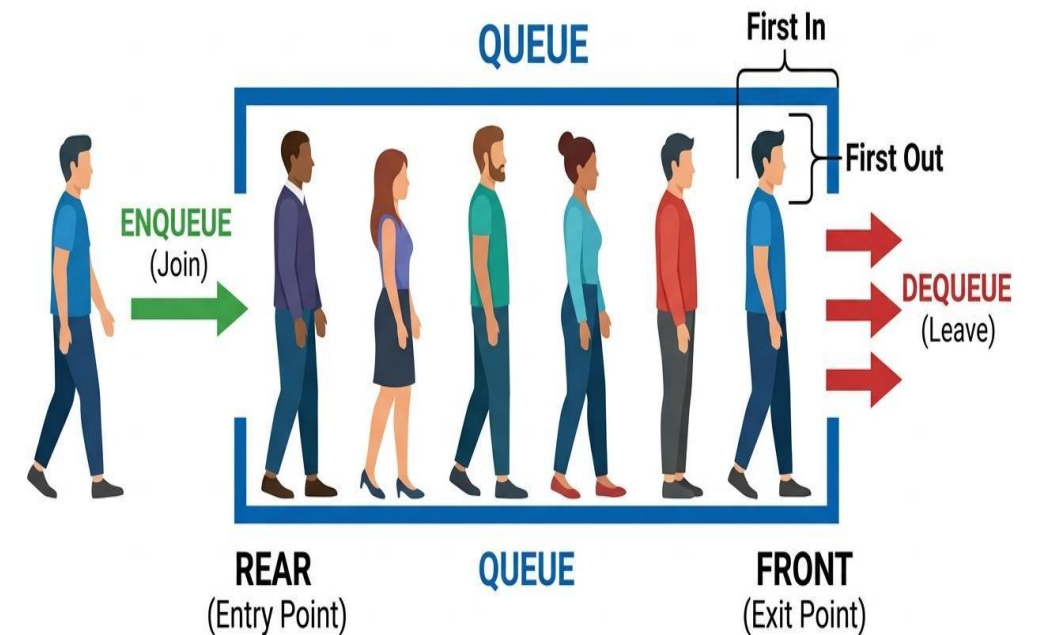
Why Queue is Efficient?

- **Enqueue:** $O(1)$
- **Dequeue:** $O(1)$
- **Peek:** $O(1)$

Advantages:

- Maintains **First-Come, First-Served** order.
- Provides fair service to passengers.
- Fast insertion and deletion.
- Easy to implement using **Arrays or Linked Lists**.
- Prevents disorder in passenger processing.

FIFO PRINCIPLE (FIRST-IN-FIRST-OUT) OF A QUEUE



CONCLUSION

- **Queue** is the most appropriate data structure for a ticket counter.
- It follows the **FIFO** principle.
- Passengers are processed in the same order in which they arrive.
- Queue provides **fairness, simplicity, and efficiency**.

TECHNICAL PRESENTATION

Doubly Linked List: Dynamic Music Playlist Management

Presented by: B. Siva Parvathi

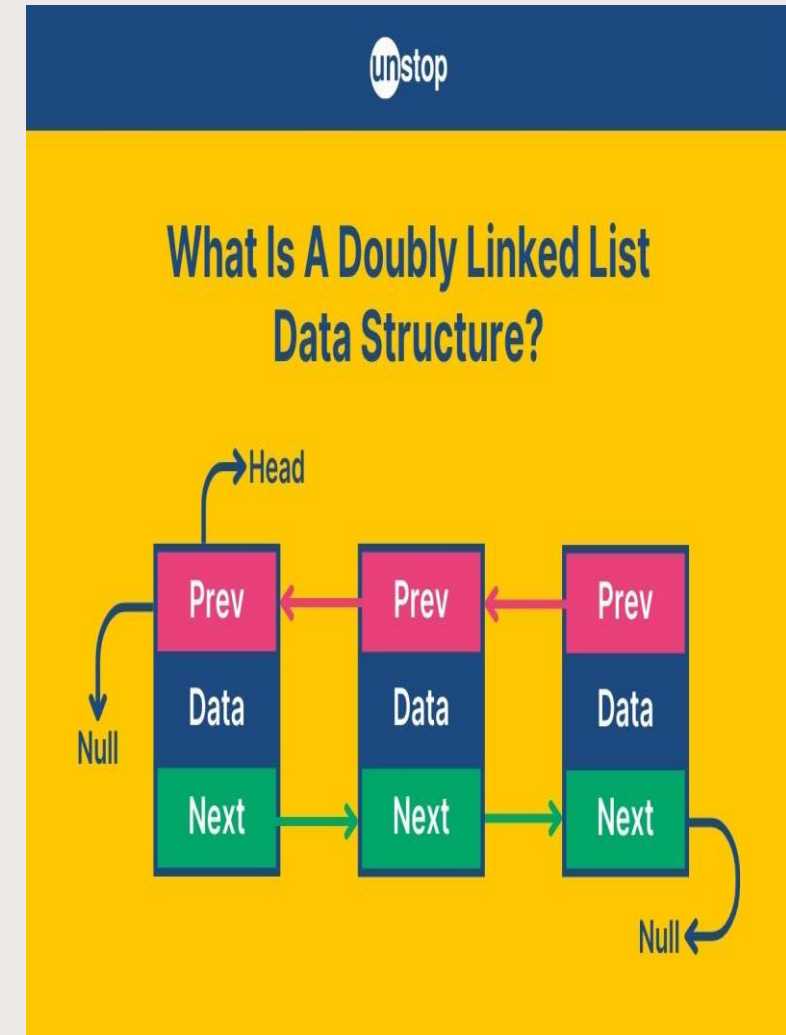
Reg no:192525194

Department: Artificial Intelligence & Machine Learning

Course: Data Structures(CSA0302)

INTRODUCTION

- A music streaming application needs to manage a playlist dynamically.
- Songs may be **added, removed, or reordered** at any time.
- A suitable data structure should provide **flexibility and efficient operations**.
- A **Doubly Linked List** is an appropriate choice for this application.
- It allows songs to be accessed and modified through links between nodes.



PROBLEM SCENARIO

Dynamic Music Playlist

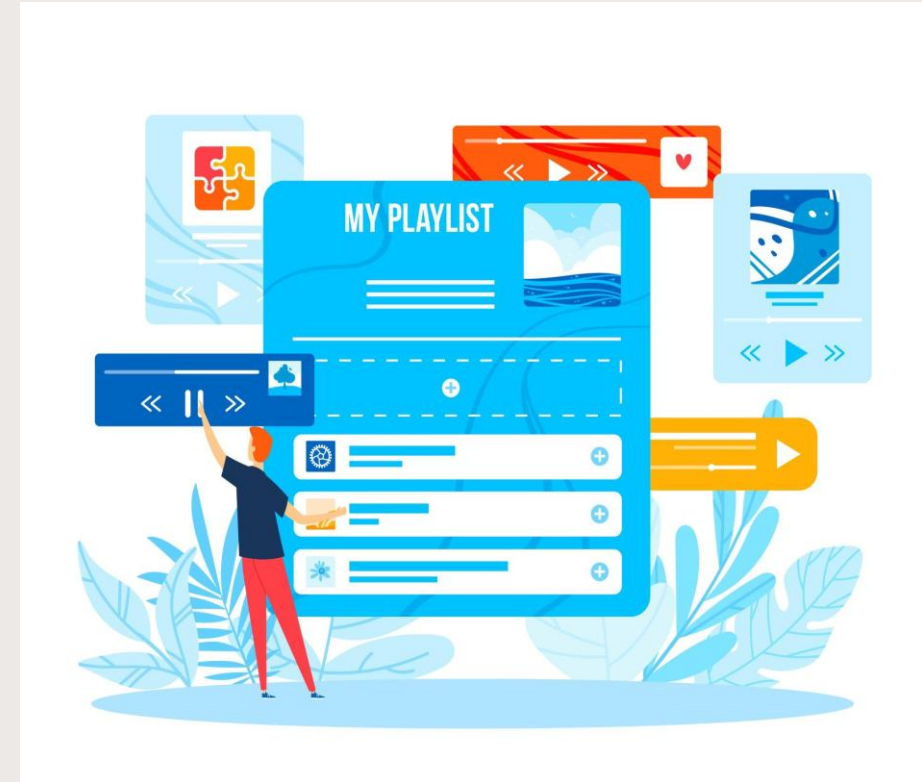
A playlist may contain:

Song A → Song B → Song C → Song D

The user can:

-  Add a new song
-  Remove an existing song
-  Reorder songs
-  Move to the next song
-  Move to the previous song

Therefore, the data structure must support **dynamic changes** efficiently.



PROPOSED DATA STRUCTURE

Doubly Linked List

A **Doubly Linked List** consists of nodes where each node contains:

[Previous | Song Data | Next]

Example:

NULL $\leftarrow$ [Song A] $\rightleftarrows$ [Song B] $\rightleftarrows$ [Song C] $\rightleftarrows$ [Song D] $\rightarrow$ NULL

- Previous:** Points to the previous song.
- Data:** Stores song information.
- Next:** Points to the next song.

This allows movement in **both forward and backward directions**.

PLAYLIST OPERATIONS

+ Adding a Song

- A new node is created.
- Links are updated to insert the song at the required position.

✗ Removing a Song

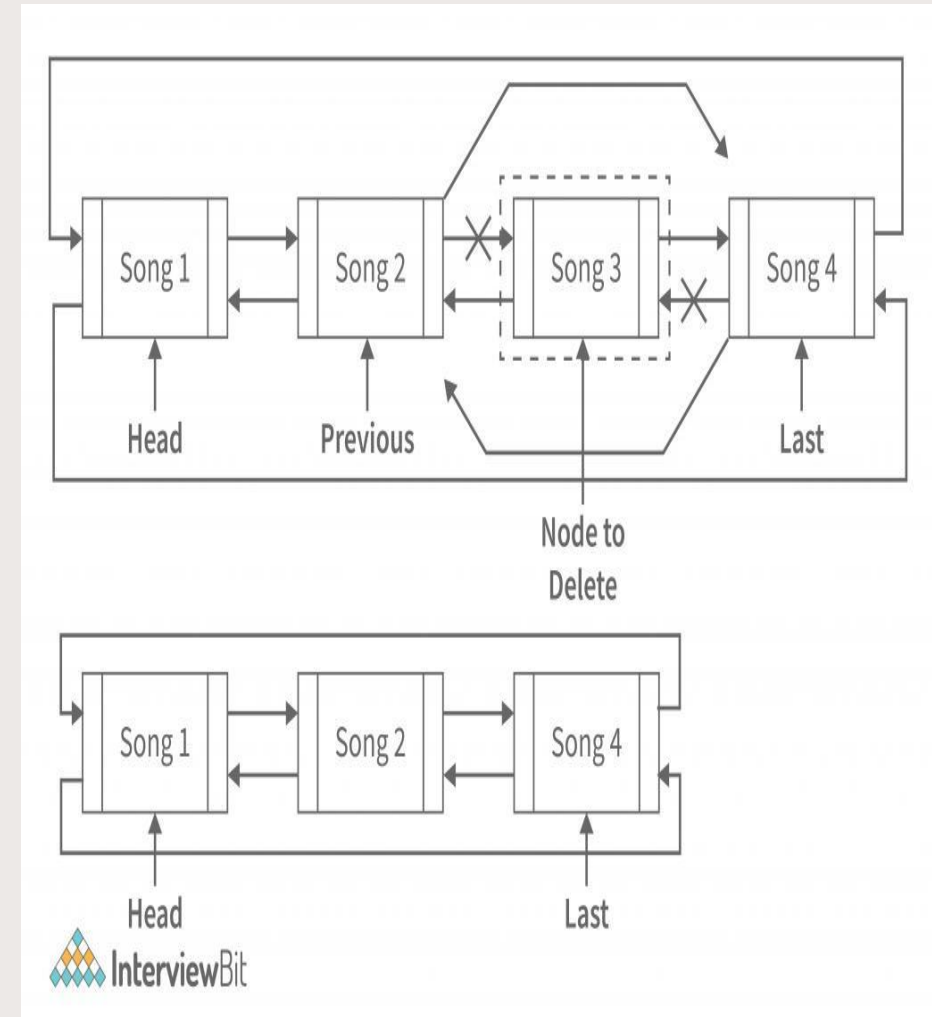
- The selected song node is removed.
- Previous and next links are updated.

↻ Reordering

- Nodes can be rearranged by changing their links.
- Songs can be moved to different positions in the playlist.

⏮ / ⏭ Navigation

- **Next** → Move using the Next pointer.
- **Previous** → Move using the Previous pointer.



ADVANTAGES

- Dynamic size; no fixed capacity required.
- Efficient insertion and deletion.
- Supports forward and backward navigation.
- Suitable for frequent playlist modifications.
- Easy to reorder songs by changing links

CONCLUSION

- A **Doubly Linked List** is well suited for a dynamic music playlist.
- It provides **flexibility** because songs can be added, removed, and reordered dynamically.
- It supports **forward and backward navigation**, which is useful for **Next** and **Previous** song controls.
- It provides efficient insertion and deletion when the required node is known.

Thank
you!